

C++ Coroutine TS Issues

Doc. no.	P0664R2
Revises	P0664R1
Date:	Revised 2018-03-08 at 14:25:00 UTC
Project:	Programming Language C++
Reference:	ISO/IEC TS 22277, C++ Extensions for Coroutines
Audience:	EWG, CWG, LEWG
Reply to:	Gor Nishanov < gorn@microsoft.com >

Introduction

All proposed resolutions wording is relative to [N4736](#).

Previous issue list is [P0664R1](#).

Table of content

- Evolution issues: [24](#) [25](#) [26](#) [27](#)
- CWG issues: [28](#)
- LEWG/LWG issues: [29](#)

Evolution/Core Issues

[24](#). The specification of initial suspend point does not correctly captures the intent

Section: 11.4.4 [dcl.fct.def.coroutine] **Status:** Active **Submitter:** Gor Nishanov **Opened:** 2018-03-08 **Last modified:** 2018-03-08

Issue:

Common use of the `initial_suspend` point in asynchronous coroutines is to suspend the coroutine during the initial invocation and post a request to resume the coroutine by a different execution agent. If an execution agent at a later point cannot resume the coroutine, for example, because it is being shutdown, an error will be reported to a coroutine by resuming the coroutine and subsequently throwing an exception from `await_resume`.

Currently, the invocation of `initial_suspend` is specified in [dcl.fct.def.coroutine] as:

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try { F } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

As specified, an exception from `await_resume` of `initial_suspend` will be thrown outside of the try-catch and will not be captured by `p.unhandled_exception()` and whoever waits for eventual completion of a coroutine will never learn about its completion.

This is specification defect. The intent is to capture an exception thrown by `await_resume` of an awaitable returned by `initial_suspend` within the try-catch enclosing of the user authored body `F`.

Proposed resolution:

Add underlying text to paragraph 11.4.4/3 as follows:

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try {
        F
    } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

, except that any exception thrown after the *initial suspend point* and before the flow of execution reaches *F* also results in entering the *handler* of the *try-block*.

25. Allow unhandled exception escape the user-defined body of the coroutine and give it well defined semantics

Section: 11.4.4 [dcl.fct.def.coroutine] **Status:** Active **Submitter:** Eric Niebler **Opened:** 2018-03-10 **Last modified:** 2018-03-10

Issue:

Currently an unhandled exception can never escape the user-authored body of the coroutine.

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    try {
        F // user authored body
    } catch(...) { p.unhandled_exception(); }
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

An exception from `F` is captured by the try-catch and a customization point `unhandled_exception` is called where, typically, an `exception_ptr` is created and propagated to the consumer awaiting on async task, or, in case of a generator, will be stored in the iterator and be delivered to the user when they dereference the iterator.

Though the current behavior is perfectly reasonable for asynchronous scenarios, it is sub-optimal for synchronous generators. Capturing an exception, storing it in an `exception_ptr` and then rethrowing the exception during, say, iterator's `operator++` is a needless work if the desired behavior is to let the exception propagate to the caller performing `operator++` on the iterator of a generator.

The proposed resolution is to make `unhandled_exception` customization point optional and give well defined behavior to the case when exception escapes user-authored body of the coroutine.

Proposed Wording:

TBD

26. Relax requirements on a `coroutine_handle` passed to `await_suspend`

Section: 8.3.8 [expr.await] **Status:** Active **Submitter:** Gor Nishanov **Opened:** 2018-03-10 **Last modified:** 2018-03-10

Issue:

One of the implementation strategies for coroutines is to chop original function into as many functions (parts) as there are suspend points. In that case, it is possible for a compiler create a unique per suspend per function `coroutine_handle` which resume and destroy members can be direct calls to corresponding parts of the function.

Though no compiler is doing it now, we can allow implementors to experiment with this approach by relaxing the requirement on the `coroutine_handle` passed to `await_suspend`.

Proposed wording:

Add underlined text to 8.3.8/3.5:

(3.5) — `h` is an object of type convertible to `std::experimental::coroutine_handle<P>` referring to the enclosing coroutine.

27. Make suspension in dynamic initializers of static and `thread_local` local variables ill-formed

Section: 8.3.8/2 [expr.await] **Status:** Active **Submitter:** Richard Smith **Opened:** 2018-03-25 **Last modified:** 2018-03-25

Proposed wording:

Add underlined text to 8.3.8/2:

An *await-expression* shall appear only in a potentially-evaluated expression within the *compound-statement* of a *function-body* outside of a *handler* (Clause 18). In a declaration-statement or in the simple-declaration (if any) of a for-init-statement, an *await-expression* shall appear only in an initializer of that declaration-statement or simple-declaration unless it is used to initialize a block-scope variable with static or thread storage duration.

CWG issues

28. Simplify stable name for Coroutines to be [def.coroutine]

Section: 11.4.4 [dcl.fct.def.coroutine] **Status:** Active **Submitter:** Gor Nishanov **Opened:** 2018-03-10 **Last modified:** 2018-03-10

Proposed wording:

[~~dcl.fct.~~def.coroutine]

LEWG/LWG issues

29. Absence of `const` on `coroutine_handle::resume()` makes lambdas verbose

Section: 21.11.2 [coroutine.handle] **Status:** Active **Submitter:** (Many) **Opened:** 2018-03-08 **Last modified:** 2018-03-08

Issue:

During LEWG/LWG review of Coroutine TS, `const` on `coroutine_handle::resume()` was removed. While it does follow LEWG/LWG convention of only putting `const` on members that are safe to call concurrently from multiple threads, it does make writing lambdas capturing `coroutine_handle` more verbose.

used to write: `[h]{ h.resume();}`

now have to write: `[h]__() mutable { h.resume();}`

Customers complained and I have recorded this as an issue.